IT1244

Artificial Intelligence: Technology & Impact

Week 7 Tutorial 5

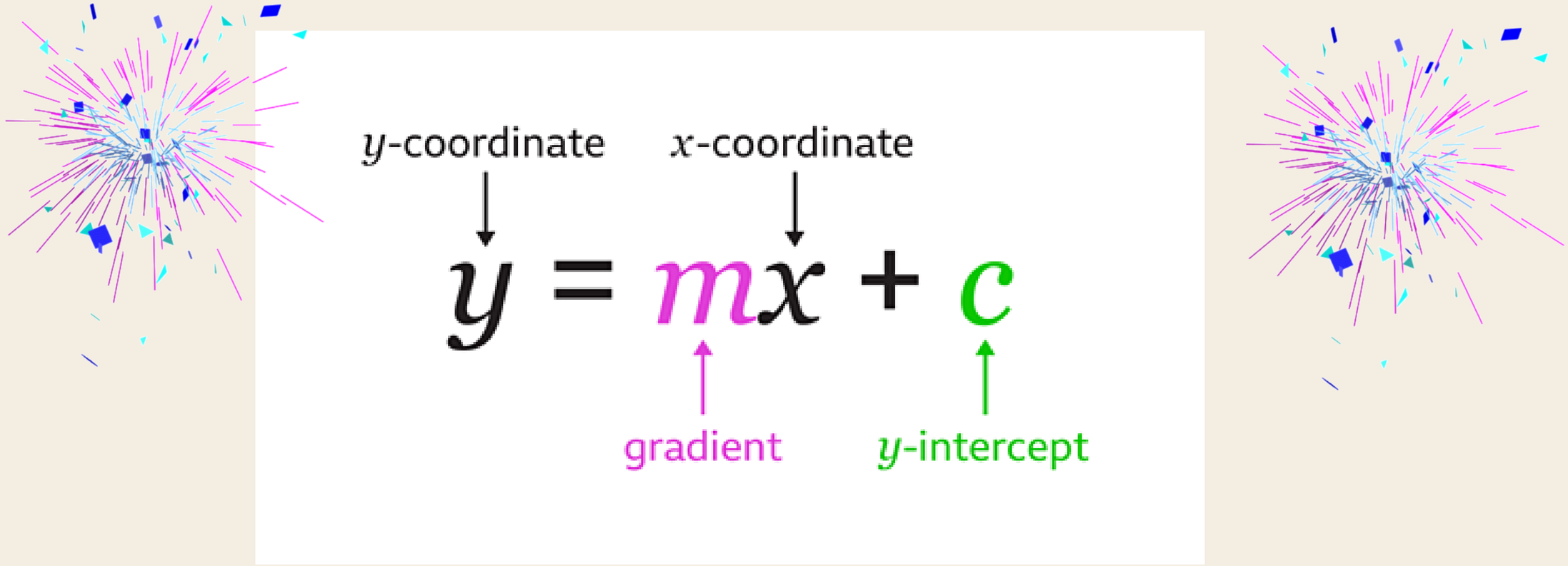
TODAY'S AGENDA

1. Recap of concepts
2. Discussion of tutorial questions
3. Pollev Questions

RECAP

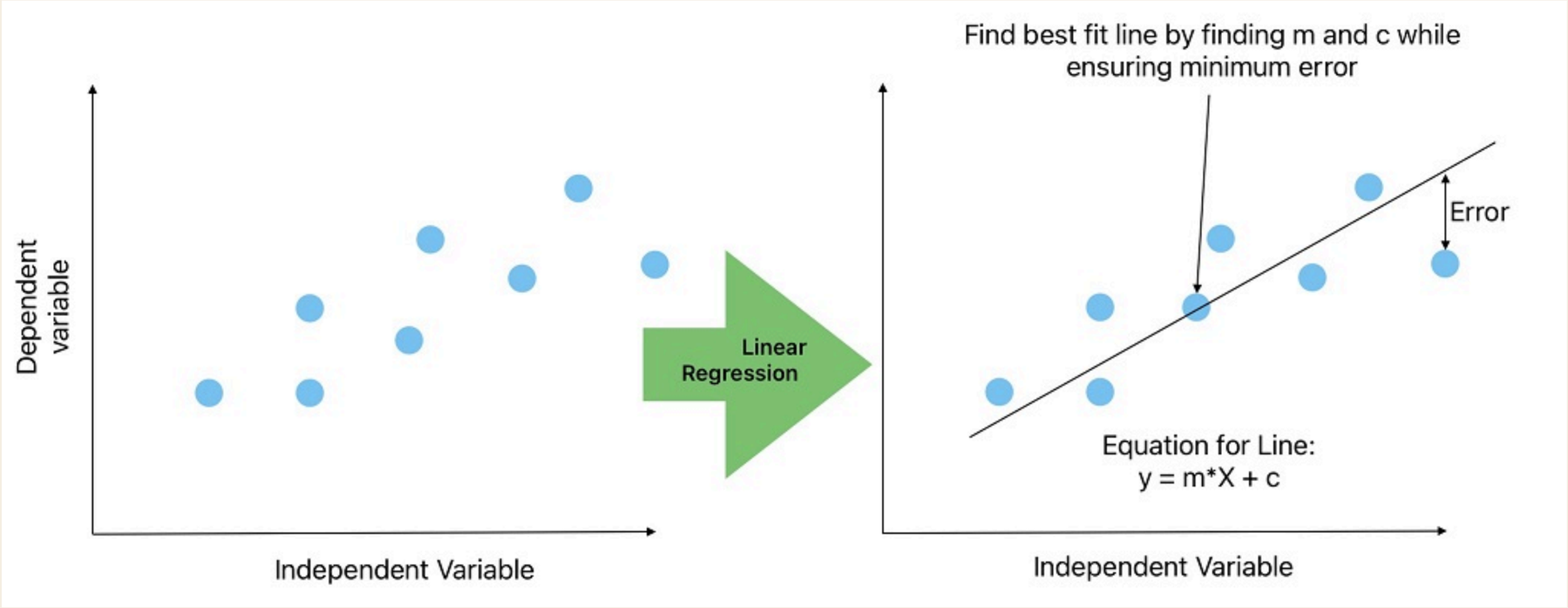
LINEAR REGRESSION

What’s the equation of a straight line?



LINEAR REGRESSION

A supervised learning algorithm that models the relationship between a continuous response variable (y) and one or more predictors (x) by fitting a linear model (a line, plane, or hyperplane depending on the number of predictors) to the data.



LINEAR REGRESSION

Same idea, just terms a little different

$$\hat{y}(x) = w_0 + w_1 x$$

y_hat(x): predicted y value when given x

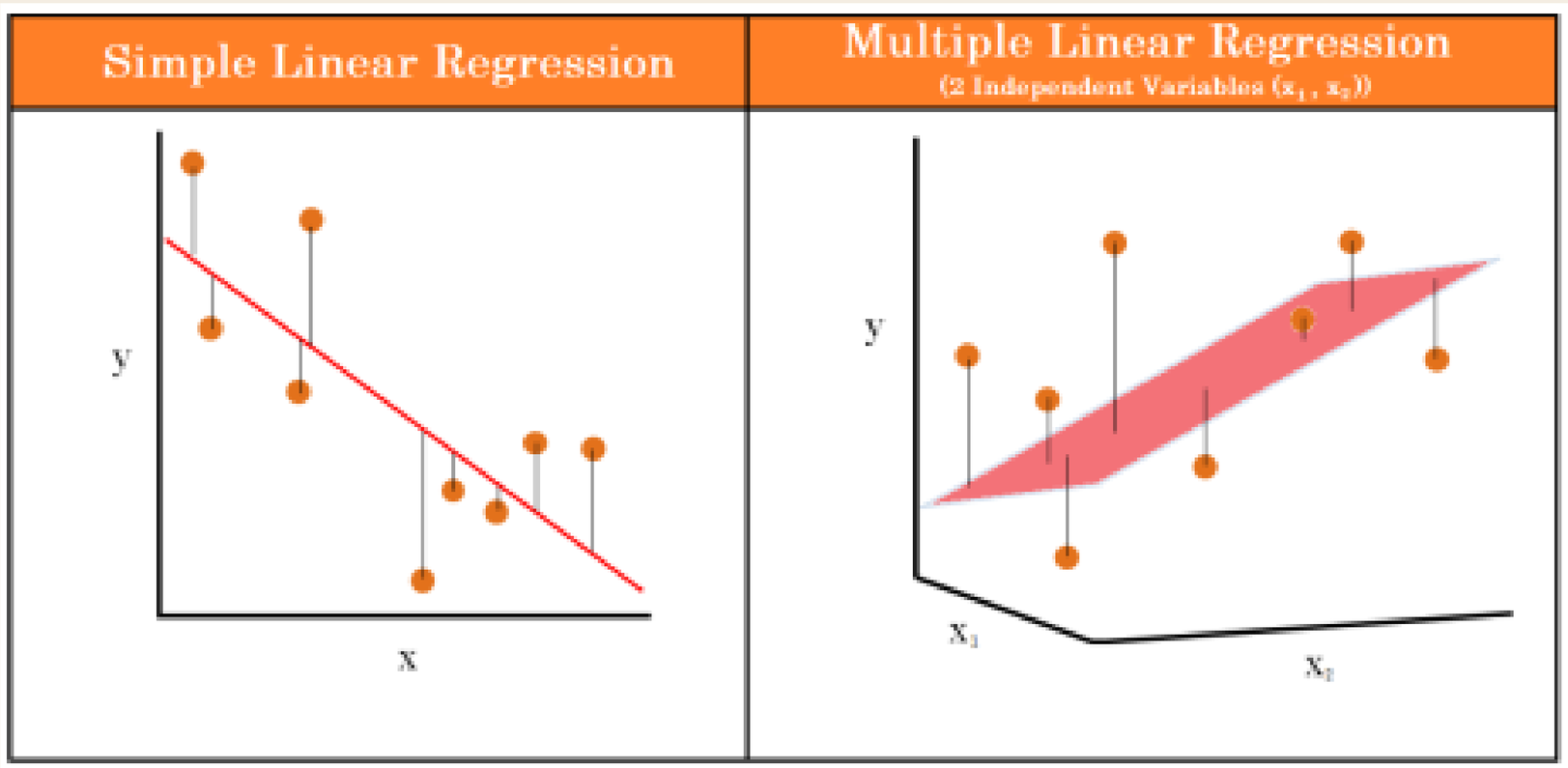
w0: Bias / Intercept

w1: Weights / Coefficients / Slope

MULTIPLE LINEAR REGRESSION

We can have many weights:

$$\hat{y}(x) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_m x_m$$



ERROR FUNCTION

Average squared differences between actual values and predicted values.

aka (scaled) Mean Squared Error MSE

$$E(w_0, w_1) = \frac{1}{2N} \sum_{i=1}^{N} \left(y^{(i)} - \hat{y}\left(x^{(i)}\right) \right)^2$$

N - *number of data points*

$y^{(i)}$ - *y value for the i-th data point*

$\hat{y}\left(x^{(i)}\right) = w_0 + w_1 x^{(i)}$

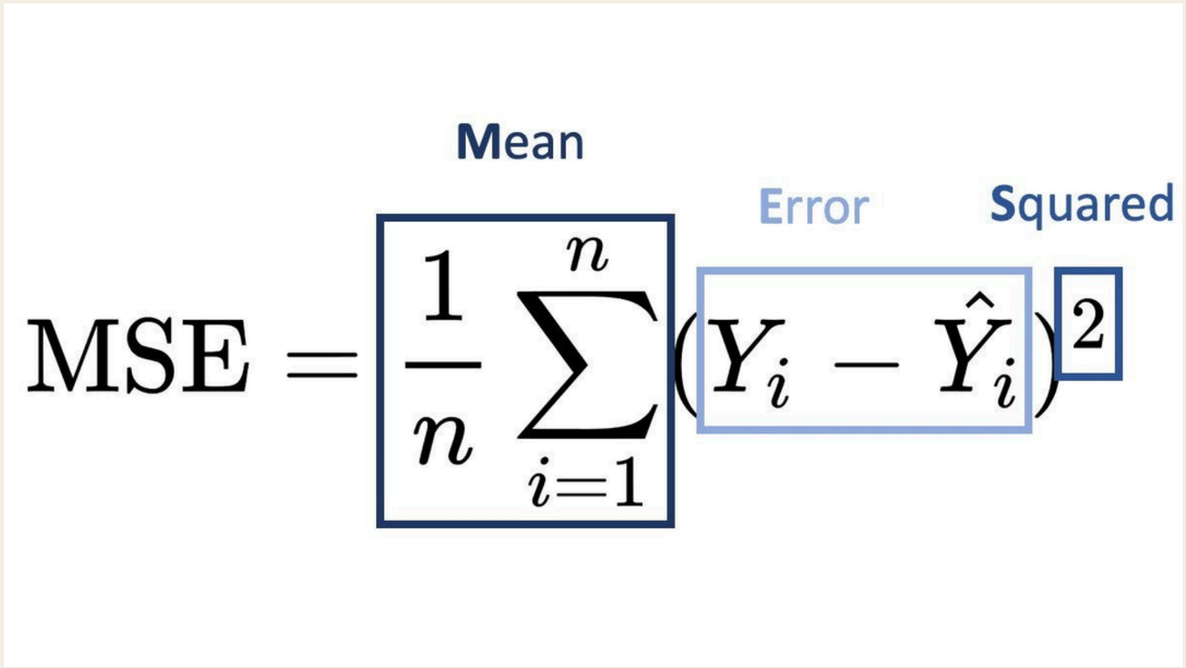
$x^{(i)}$ - *x value for the i-th data point*

Fun Fact:

Error function for linear regression is a strict convex function - meaning that you can **always find a unique minimum to it**

ERROR FUNCTION

WAIT! MSE is 1/N, Why is the error function 1/2N 😕😕

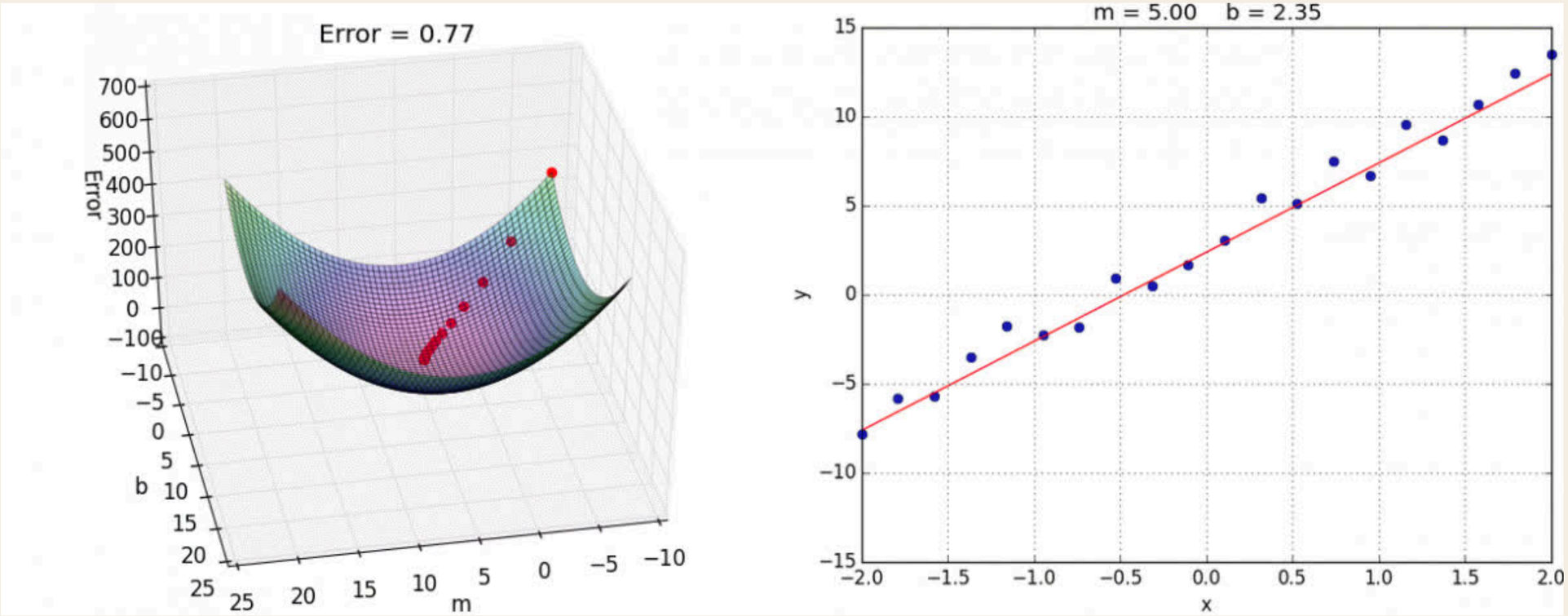


2N is what our course decides to define the average to make the partial derivatives of the error function simpler.

- Hence “scaled” MSE

GRADIENT DESCENT ALGORITHM

Recall that we are interested in finding the optimal parameters (w0* and w1*) for our simple regression model



this is a gif!

GRADIENT DESCENT STEPS

1. Initialize w0, w1
   - Either choose specific values (eg. 0) / randomise starting position

2. Start with a small learning rate, L (eg. 0.01)

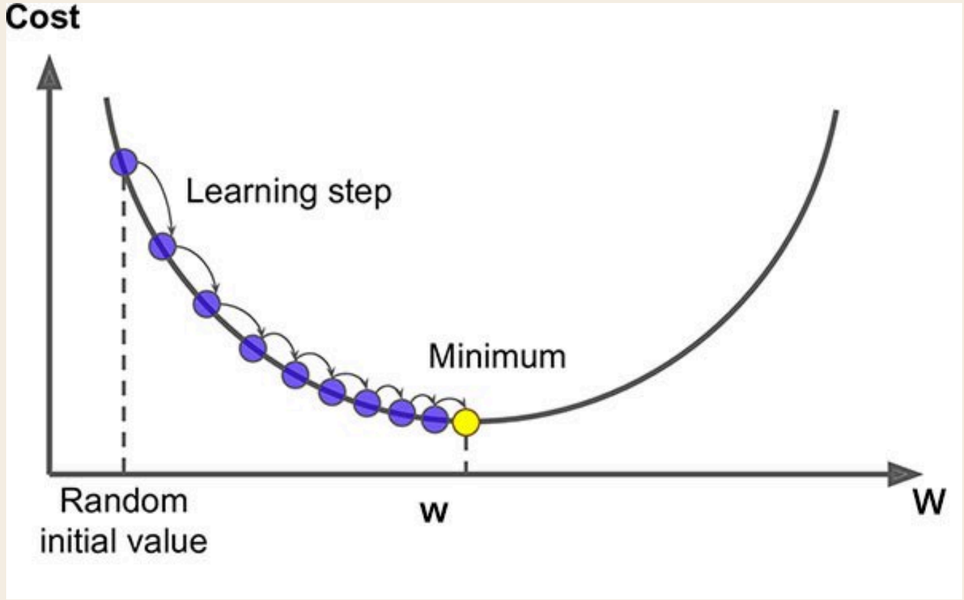
3. Calculate the partial derivative of the error function wrt w0 and w1

$$\frac{\partial E}{\partial w_j} = -\frac{1}{N}\sum_{i=1}^{N}\left(y^{(i)} - \hat{y}\left(x^{(i)}\right)\right) \cdot x_j^{(i)}$$

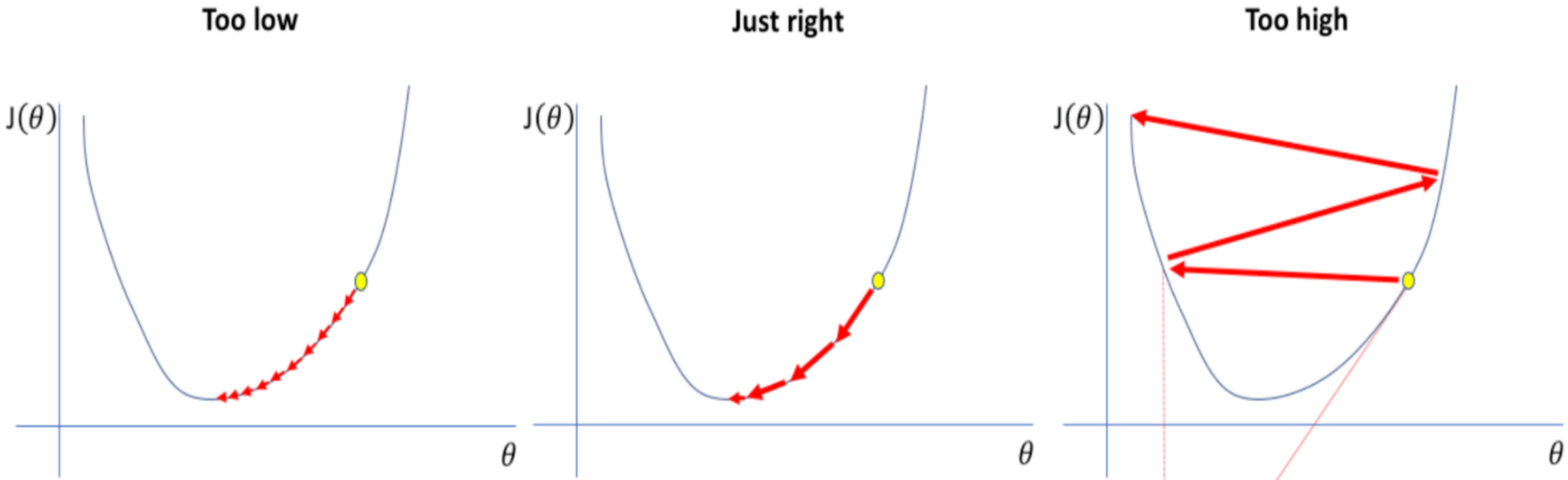
4. Update the current weights (w0, w1)

$$w_{\mathrm{i}} = w_i - L\frac{\partial}{\partial w_{\mathrm{i}}}E(w_0, w_1, \ldots, w_{\mathrm{n}})$$

5. Repeat steps and until **convergence**
   - the error function is minimised to small value
   - number of iterations



HOW TO CHOOSE L?



PREDICTING NEW DATA

Once we found all optimal values for the parameters (w*0, w*1, …, w*n), we have our fitted equation:

$$\hat{y}(x) = w_0^* + w_1^* x_1 + w_2^* x_2 + \cdots + w_m^* x_m$$

Just sub new data (x) in to get it’s y_hat!

TUTORIAL QUESTIONS

TUTORIAL QUESTION 1A

Initialize the values of [w0, w1] to [0, 0] and run the Gradient Descent algorithm for 3 iterations and show the values of [w0, w1] at the end of each iteration. Assume the learning rate L = 0.01.

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

TUTORIAL QUESTION 1A

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

JUST FOLLOW THESE STEPS

① $\hat{y} = w_0 + w_1 x$

② $\Delta w_0 = \frac{1}{n} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})$

$\Delta w_1 = \frac{1}{n} \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)}) x^{(i)}$

③ $w_0 = w_0 + L \Delta w_0$

$w_1 = w_1 + L \Delta w_1$

TUTORIAL QUESTION 1A

[w0, w1] = [0,0] | L = 0.01 | 3 iterations

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

Iteration 1

Step 1: $\hat{y}^{(i)} = w_0 + w_1 x^{(i)}$

since $w_0 = w_1 = 0$, all $\hat{y} = 0$

Step 2: Δw_0 and Δw_1

$$\Delta w_0 = \frac{1}{6}(4.5 + 12 + 14.5 + 2 + 7 + 27)$$

$$= 67/6$$

$$\Delta w_1 = \frac{1}{6}(4.5(3) + 12(6) + 14.5(7) + (2)(2) + (4)(7) + (12)(27))$$

$$= 543/6 = 90.5$$

TUTORIAL QUESTION 1A

[w0, w1] = [0,0] | L = 0.01 | 3 iterations

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

Iteration 1

Step 3 : Update w0 and w1

$w_0 = 0 + 0.01\left(\frac{67}{6}\right) = 0.11167$

$w_1 = 0 + 0.01(90.5) = 0.905$

w0 and w1 after iteration 1:

[0.11167, 0.905]

TUTORIAL QUESTION 1A

[w0, w1] = [0,0] | L = 0.01 | 3 iterations

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

Repeat for Iteration 2 and 3:

Iteration 1: [0.111667, 0.905]

Iteration 2: [0.17093,1.41452]

Iteration 3: [0.20073, 1.70159]

TUTORIAL QUESTION 1A

Code for this cause I was in a good mood :D

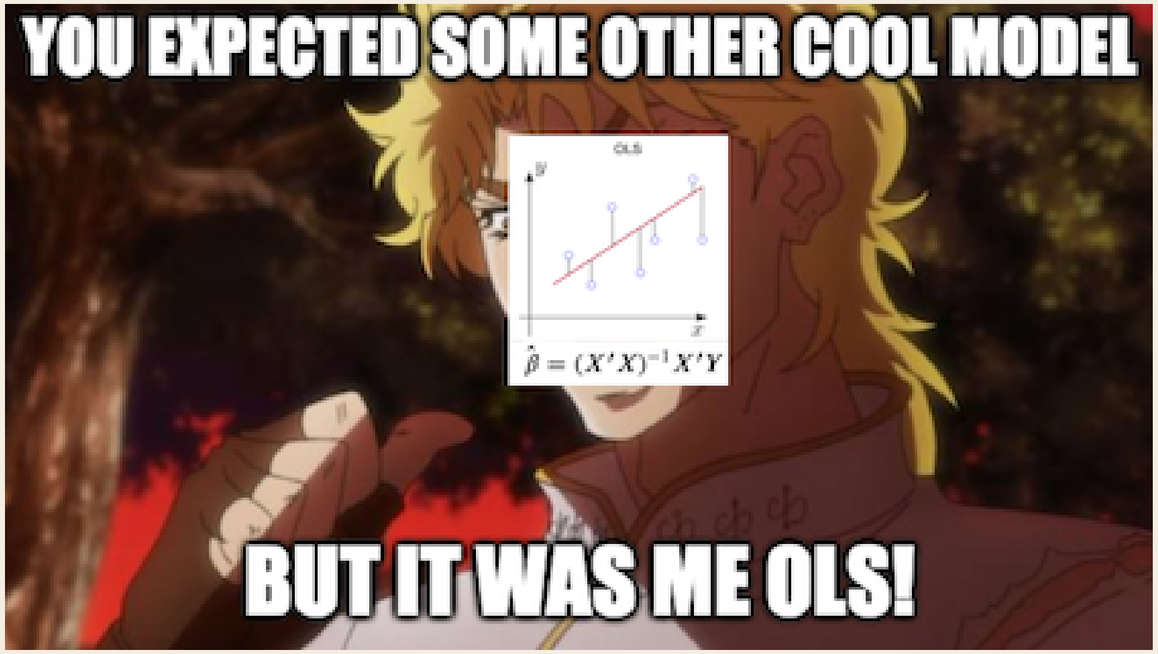
```python
import pandas as pd

# Gradient Descent Algorithm for Linear Regression

def gradient_descent(df, w0, w1, L, N):

    # Step 0: Get number of rows in the dataframe
    m = df.shape[0]

    for _ in range(N):

        # Step 1: Calculate y_hat
        df['y_hat'] = w0 + w1 * df['x']

        # Step 2: Calculate delta for w0 and w1
        # Note that this is the derivative of the error/cost function wrt w0 and w1

        df['delta_w0'] = df['y'] - df['y_hat']

        df['delta_w1'] = (df['y'] - df['y_hat']) * df['x']

        delta_w0 = 1/m * df['delta_w0'].sum()
        delta_w1 = 1/m * df['delta_w1'].sum()

        # Step 3: Update w0 and w1
        w0 = w0 + L * delta_w0
        w1 = w1 + L * delta_w1

        print(f'w0: {round(w0,5)}, w1: {round(w1,5)}')

    return w0, w1
```

```
w0: 0.11167, w1: 0.905
w0: 0.17093, w1: 1.41452
w0: 0.20073, w1: 1.70159
```

TUTORIAL QUESTION 1B

Find out the parameters w0 and w1 and write down the best-fit line equation.

Note: Since this is single variable linear regression, the parameters w0 and w1 can also be found using formulas and **don’t need to run Gradient Descent algorithm.**



TUTORIAL QUESTION 1B

Find out the parameters w0 and w1 and write down the best-fit line equation.

Closed-form formula:

$$w1 = \frac{\sum_{i=1}^{N}(x_i - mean(x))\,(y_i - mean(y))}{\sum_{i=1}^{N}(x_i - mean(x))^2}$$

$$w0 = mean(y) - w_1\, mean(x)$$

Simply sub the values inside the closed-form formula to get w0 and w1

```
w0: -3.0, w1: 2.5
```

Best-fit Line Equation:

```
Best fit line eqn: y = -3.0 + 2.5x
```

TUTORIAL QUESTION 1B

Find out the parameters w0 and w1 and write down the best-fit line equation.

```python
import pandas as pd

data = {'x': [3,6,7,2,4,12],
        'y': [4.5, 12, 14.5, 2, 7, 27]}

df = pd.DataFrame(data)

# Closed Form Solution for Linear Regression

def closed_form_solution(df):

    # Get number of rows in the dataframe
    m = df.shape[0]

    # Calculate w1 and w0 using the closed form solution formula
    w1 = ((df['x'] - df['x'].mean()) * (df['y'] - df['y'].mean())).sum() / ((df['x'] - df['x'].mean())**2).sum()

    w0 = df['y'].mean() - w1 * df['x'].mean()

    return w0, w1

w0, w1 = closed_form_solution(df)

print(f'w0: {round(w0,2)}, w1: {round(w1,2)}')

print(f'Best fit line eqn: y = {round(w0,2)} + {round(w1,2)}x')
```

Also in code if interested!

TUTORIAL QUESTION 1C

When x = 7, predict the value of y and the error associated with it.

	x	y
1	3	4.5
2	6	12
3	7	14.5
4	2	2
5	4	7
6	12	27

```
Best fit line eqn: y = −3.0 + 2.5x
```

Step 1: Sub x into best fit eqn

y = -3.0 + 2.5(7) = **14.5**

Step 2: Error = y_hat - y

14.5 - 14.5 = 0

Because error (residual) is 0, the data point lies exactly on the regression line

TUTORIAL QUESTION 1D

What is the predicted value of y for a new data x = 10.

```
Best fit line eqn: y = -3.0 + 2.5x
```

<u>Sub new x into best fit eqn</u>

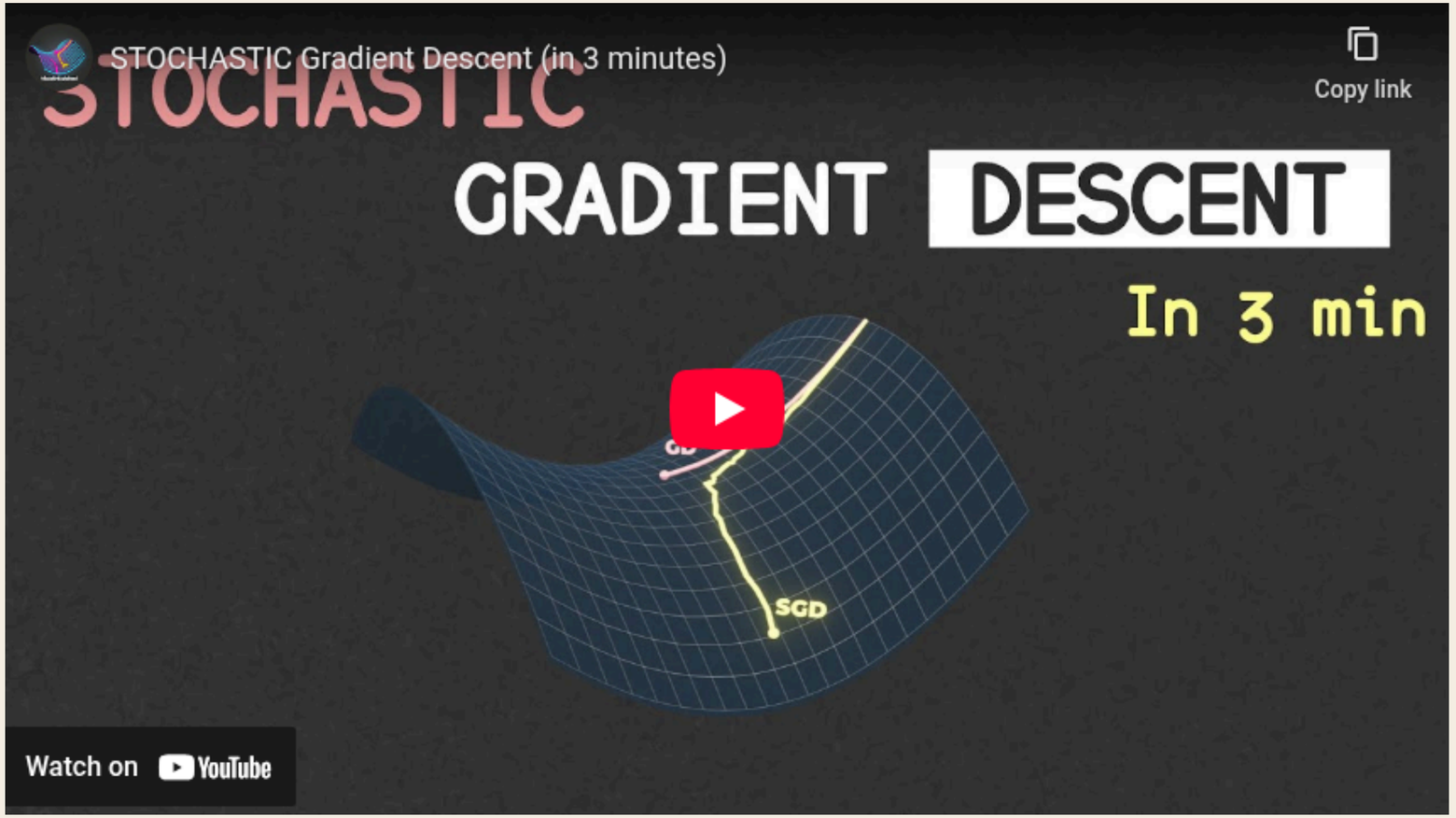
y = -3.0 + 2.5(10) = **22.0**

<u>Interpolation or Extrapolation?</u>

Interpolation as x = 10 lies within the observed range of x-values (2 to 12)

TUTORIAL QUESTION 2A

How does the Stochastic Gradient descent works?



TUTORIAL QUESTION 2A

How does the Stochastic Gradient descent works?

Stochastic Gradient Descent:

- Pick a **random** instance in the training data at every step
- Compute the loss value & gradient based on that **one instance**
- Update parameters

TUTORIAL QUESTION 2B

Gradient Descent vs Stochastic Gradient Descent difference

```
BATCH GRADIENT DESCENT (GD)
---------------------------
input: data (X, y), learning_rate η, max_epochs E
initialize w randomly (or zeros)

for epoch = 1 to E:
    # 1) compute predictions for ALL samples
    y_hat = f(X, w)

    # 2) compute average gradient over ALL samples
    grad = (1/N) * Σ_{i=1..N} ∇w loss( y[i], f(x[i], w) )

    # 3) update once per epoch (or per full pass)
    w = w - η * grad

return w
```

```
STOCHASTIC GRADIENT DESCENT (SGD)
---------------------------------
input: data (X, y), learning_rate η, max_epochs E
initialize w randomly (or zeros)

for epoch = 1 to E:
    shuffle the training examples

    for i = 1 to N:
        # 1) compute prediction for ONE sample
        y_hat_i = f(x[i], w)

        # 2) compute gradient using ONLY that sample
        grad_i = ∇w loss( y[i], y_hat_i )

        # 3) update immediately (N updates per epoch)
        w = w - η * grad_i

return w
```

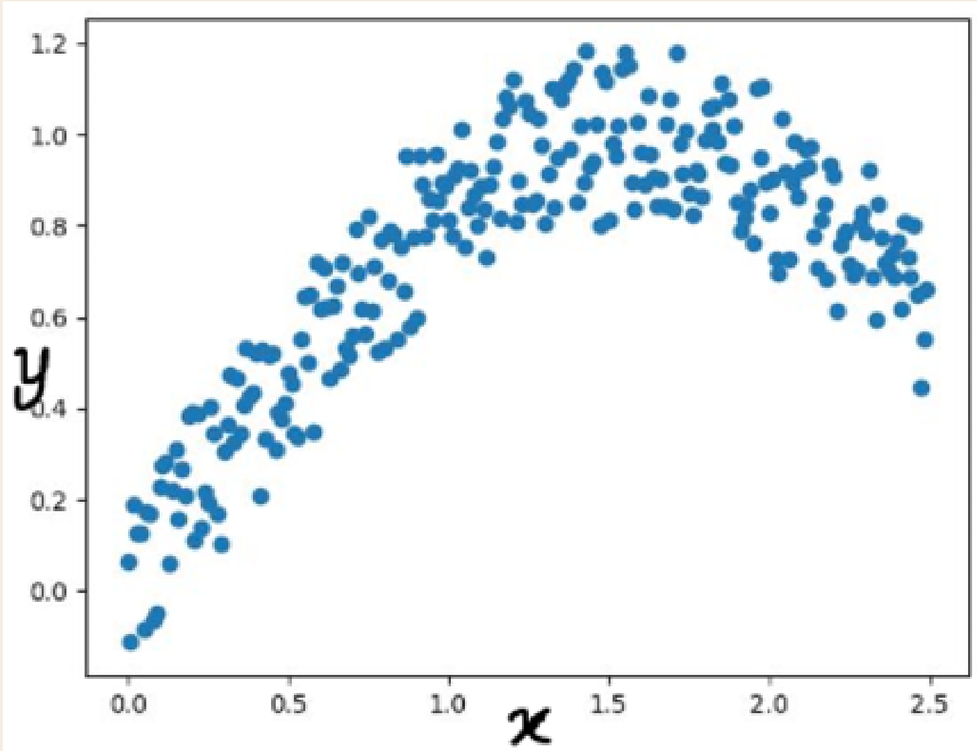
TUTORIAL QUESTION 2B

Gradient Descent vs Stochastic Gradient Descent vs Mini-batch GD

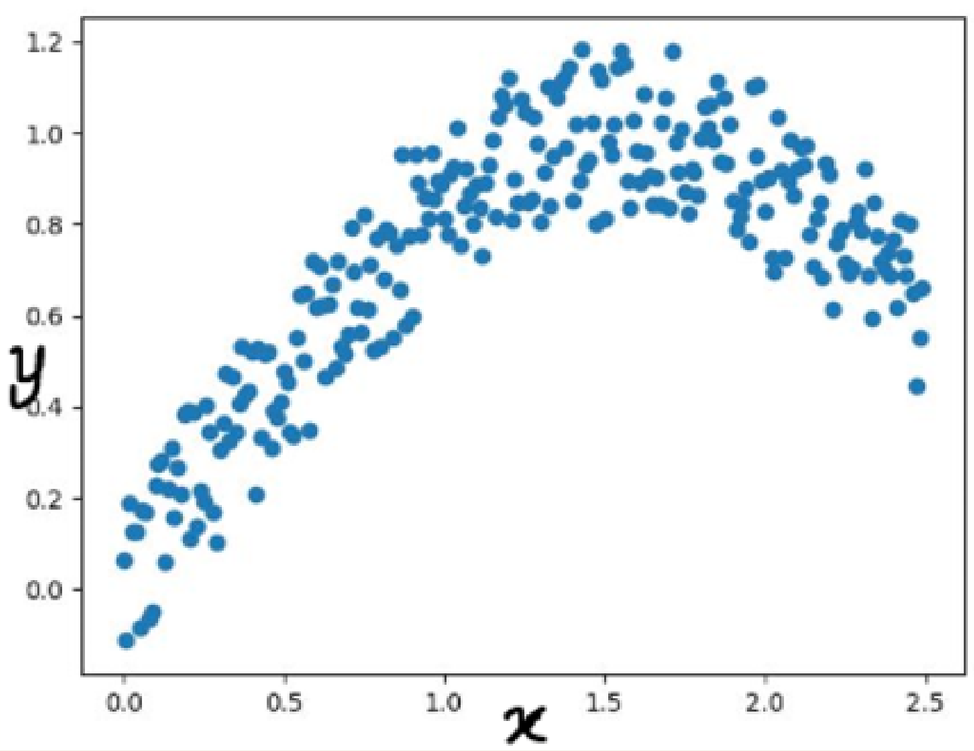
Feature	Batch GD	Stochastic GD	Mini-Batch GD
Update Frequency	Once per epoch	Once per data point	Once per mini-batch
Stability	Smooth and stable	Noisy	Balanced
Speed	Slow for large datasets	Fast for large datasets	Moderate
Memory Requirements	High (entire dataset)	Low (single data point)	Moderate
Convergence	Exact minimum	Approximates minimum	Balanced
Use Case	Small datasets	Online/streaming data	Deep learning, large datasets

TUTORIAL QUESTION 3

If your data is not plotted to fit a straight line, then how would you apply linear regression. For example, the scatter plot shown below does not fit a straight line. How would you design a linear regression for those data?



TUTORIAL QUESTION 3



- x and y do not have a linear relationship
- so obviously, linear regression doesn't work here
- We can use **polynomial regression** instead of linear regression

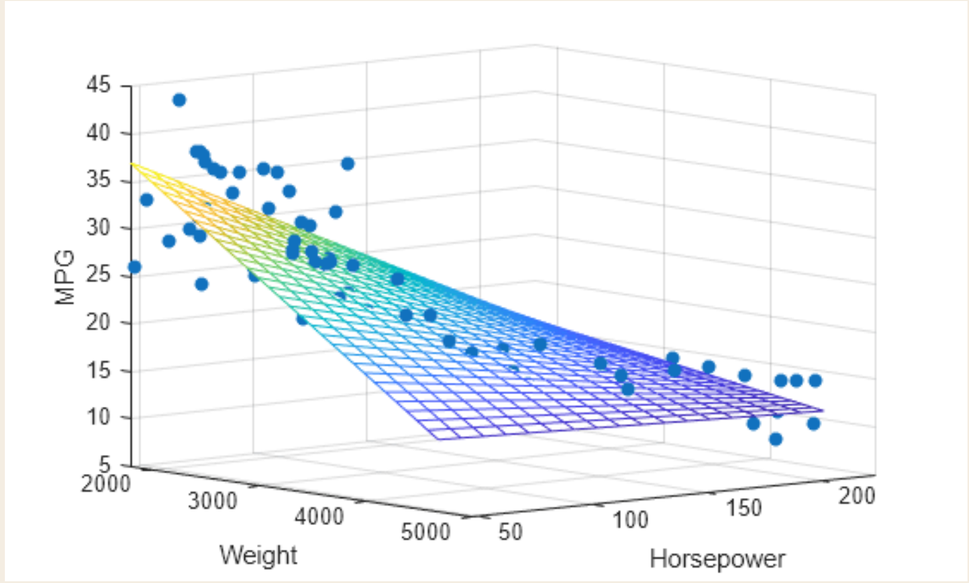
https://www.datatechnotes.com/2018/06/polynomial-regression-fitting-in-python.html

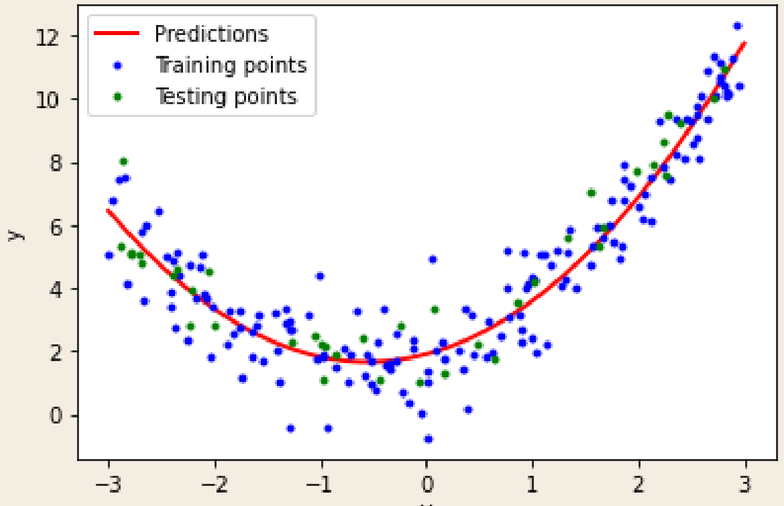
TUTORIAL QUESTION 3

Linear Regression Types

DailyDoseofDS.com

Algorithm	Description	Equation/Loss Function
Simple Linear Regression	One independent (x) and one dependent (y) variable	$\hat{y} = wx + b$
Polynomial Linear Regression	Polynomial features ($x, x^2, \ldots, x^n$) and one dependent (y) variable	$\hat{y} = w_1 x + w_2 x^2 + \cdots + b$
Multiple Linear Regression	Arbitrary features ($x_1, x_2, \ldots, x_n$) and one dependent (y) variable	$\hat{y} = w_1 x_1 + w_2 x_2 + \cdots + b$





which is MLR and which is PLR?

POLLEV QUESTIONS

Join by Web:

PollEv.com/kaironglee

Join by QR:

EXTRA QUESTION

x1	x2	y
9	0	25
9	1	30
10	-1	22
8	1	28
7	4	41
10	4	47
10	5	52
8	-3	8
7	2	31
8	2	33

Credits to TA Jason!

Initialise the values of [w0, w1, w2] to [0, 0, 0] and run the Gradient Descent algorithm for 3 iterations. Show the values of [w0, w1, w2] at the end of each iteration, assuming that L = 0.01.

This will act as a practice question for you to prepare for the final, in case they ask you to perform GD on multivariable data.

EXTRA QUESTION ANSWER

x1	x2	y
9	0	25
9	1	30
10	-1	22
8	1	28
7	4	41
10	4	47
10	5	52
8	-3	8
7	2	31
8	2	33

After 1st iter	w0	w1	w2
	0.317	2.761	0.752
After 2nd iter	w0	w1	w2
	0.382104	3.319954	1.07965
After 3rd iter	w0	w1	w2
	0.39357217	3.4100535	1.30787142

Credits to TA Jason!

EXTRA READINGS

- Evaluating Linear Regression - R^2 and Adjust R^2
- Lasso and Ridge Regression
- Extensive Comparison between Batch GD and Stochastic GD
- What is Mini-Batch GD? (Middle ground of Batch GD and Stochastic GD)

NEXT WEEK

Logistic Regression & Model Validation